PyHelmholtz

version 0.1.10

A Python package for solving the 2D Helmholtz equation and benchmarking absorbing boundary methods.

Features

- Fast frequency-domain wave propagation modeling.
- Support for standard sparse direct solvers via SciPy.
- Optional high-performance nested dissection algorithms via SuiteSparse.
- Optional parallel direct solver acceleration via MUMPS.

Installation

PyHelmholtz provides three installation tiers depending on your performance needs and platform.

1. Standard Installation (Recommended)

If you only need standard solvers using SciPy's `spsolve()`, you can install PyHelmholtz instantly with no external C-library or compiler requirements:

```
pip install pyhelmholtz
```

2. With Nested Dissection Support (Optional)

To enable advanced nested dissection algorithms, the package requires `scikit-sparse`. Because this depends on underlying SuiteSparse C-libraries, it is highly recommended to install the dependencies via Conda first to avoid compilation issues (especially on Windows):

```
# Install the C-libraries and pre-compiled wrapper via Conda
conda install -c conda-forge scikit-sparse

# Install PyHelmholtz with the optional suitesparse feature flag
pip install pyhelmholtz[suitesparse]
```

3. With MUMPS Acceleration (Optional, Linux only)

For large-scale problems requiring parallel direct solvers, PyHelmholtz interfaces with MUMPS via PyMUMPS. Because this relies on compiled Fortran, MPI, and BLAS/LAPACK binaries, you must install the system MUMPS libraries first before installing the Python package:

```
# Install system MUMPS and MPI development libraries (Ubuntu/Debian)
sudo apt-get update
sudo apt-get install libmumps-dev

# Install PyHelmholtz with the optional mumps feature flag
pip install pyhelmholtz[mumps]
```

Quick Start

Here is a quick example of how to use PyHelmholtz to solve the Helmholtz equation using the default parameters as shown in Table 1:

Parameters	Values
physical domain Ω_i	[-1, 1] × [-1, 1] m^2
velocity $v(x, y)$	2.998 $\times 10^8$ m/s (homogeneous)
source type	point source
source location	(0, 0) m
source frequency f	2 GHz
source strength S	1
grid spacing h_x and h_y	0.01 m ($h_x = h_y$)

Numerical configurations	Settings
FD accuracy order	2
absorbing boundary method	second-order Engquist-Majda (EM2) (one-cell boundary layer)
total grid points, $N_x \times N_y$	203 × 203
linear solver	SuperLU (`scipy.sparse.linalg.spsolve()`)

Table 1: Default parameters and numerical configurations of the `Helmholtz` class.

```
import pyhelmholtz as ph

ho = ph.Helmholtz() # create a Helmholtz object with default parameters
ho.solve() # SciPy's spsolve() is used by default as the sparse linear solver
ho.viz() # visualize the real part of the wave field u
```

Here is another example in which several parameters were manually set. These include the grid spacing $h = 10$ m, the horizontal and vertical domain limits = $[0, 1000]$ and $[0, 500]$ respectively, the wave speed = 1000 m/s, the point source frequency = 10 Hz, and location = $[500, 250]$. In addition, the perfectly matched layer (PML) is used as the absorbing boundary method, and the fourth-order FD schemes are used in the calculation.

```
import pyhelmholtz as ph

domain = ph.Domain(h=10, limits=[0, 1000, 0, 500], v=1000)
source = ph.PointSource(freq=10, xs=500, ys=250)
ho = ph.Helmholtz(domain=domain, abm=ph.PML(), source=source, fd=ph.FD(4))

ho.solve()
ho.viz()
```

API Reference

This reference details all key classes and methods provided by `PyHelmholtz`. The library uses a modular object-oriented layout: physical domain setup, source terms, boundary conditions, and discretization operators are configured independently as sub-objects and passed into the top-level `Helmholtz` solver.

1. Domain Configuration

The `Domain` object defines the 2D computational spatial grid, spatial limits, and background wave speed profile. It also provides utility methods for embedding geometric velocity anomalies into the grid.

```
ph.Domain(limits=(-1,1,-1,1), h=0.01, v=299792458, positive_downward=False)
```

- `limits`: Bounds $(x_{min}, x_{max}, y_{min}, y_{max})$, scalar L, or 2-tuple x_{max}, y_{max}).
- `h`: Grid spacing float or (dx, dy) tuple.

- `v`: Wave speed scalar (homogeneous) or 2D NumPy array (inhomogeneous).
- `positive_downward`: Flips y-axis orientation if `True` (useful for geophysical conventions).
- **Methods:** `add_circle(center, radius, vel)`, `add_rectangle(box, vel)`, `is_homogeneous()`.

2. Wave Sources

Wave excitations are defined by instantiating sub-classes derived from the base source interface. These objects build the right-hand side source vector b for the linear system.

- `ph.PointSource(freq=2e9, xs=0.0, ys=0.0, strength=1)`
  Generates a spatial point source (Dirac delta profile) at coordinates (x_s, y_s) for target frequency `freq`.
- `ph.PlaneWave(freq=2e9, c0=299792458, strength=1, theta=0.0, xzp=0.0, yzp=0.0)`
  Generates an incident plane wave propagating at angle `theta` (in degrees) with respect to the x-axis.

3. Absorbing Boundary Conditions (ABM)

Absorbing boundary classes handle non-reflecting outer domain truncation to suppress unphysical boundary reflections.

- `ph.RenLiu(abm="EM2", n=1)`: Standard Engquist-Majda (`"EM1"`/`"EM2"`, padding $n = 1$) or Ren-Liu hybrid absorbing layer condition (`"RL1"`/`"RL2"`, $n > 1$).
- `ph.PML(abm="PML0", n=10)`: Perfectly Matched Layer method. Terminated with Dirichlet zero conditions (`"PML0"`) or hybrid EM conditions (`"PML1"`/`"PML2"`). Requires thickness $n \geq 2$.

4. Finite Difference Order

Defines spatial derivative discretization stencils.

`ph.FD(order=2)`: Specifies stencil order of accuracy.

5. Core Helmholtz Solver

The `Helmholtz` class binds together domain geometry, wave source, boundary treatment, and finite-difference scheme. It manages matrix assembly, linear system solving, data visualization, and analytic verification.

`ph.Helmholtz(domain=Domain(), source=PointSource(), abm=RenLiu(), fd=FD())`

- `build()`: Assembles the global discretized linear system matrix $\mathbf{A}$ and source vector $\mathbf{b}$, returning $(\mathbf{A}, \mathbf{b})$.
- `solve(solver="spsolve")`: Solves $\mathbf{Ax} = \mathbf{b}$ and stores the wavefield solution internally in `self.u`. Supported solver backends: `"spsolve"` (SciPy), `"mumps"` (PyMUMPS parallel), `"nested_dissection"` (SuiteSparse reordering), or `"gmres"` (Iterative GMRES with ILU preconditioning).
- `viz(data="solution", mode="real", unit="m", cmap="jet", vlim=None)`: Plots target arrays (`"solution"`, `"velocity"`, `"incident"`, `"total"`) across specified component modes (`"real"`, `"imag"`, `"abs"`).
- `analytic_solution_point_source()` / `error_norm_point_source()`: Calculates the Hankel-function analytical reference solutions for homogeneous media and returns relative L_2 error norm percentage.